Known limitations, defects, and improvement plan

Basis: static code inspection on main on October 14, 2026. The items below do not claim that a
bug was reproduced in an installation. "Observed limitation/risk" describes an explicit limit or
visible gap in the source. Priorities are recommendations, not delivery commitments.

Priority levels

- P0 -- trust blocker: risk of data loss, credential exposure, corruption, or unsafe
  installation.
- P1 -- stability: connectivity, continuous operation, audio/video, and error recovery.
- P2 -- capability: limits and incomplete integrations.
- P3 -- polish: telemetry, documentation, and experience.

Gap matrix

Priority | Area | Observed limitation | Practical impact | Recommended improvement
P1 | Companion -> Wear | The diagnostic UI says automatic reconnection is not enabled. | A locked
phone, suspended process, or Bluetooth loss may leave the watch offline. | Implement
lifecycle-aware reconnection with backoff and a foreground service/persistent notification where
allowed; test screen-off and manufacturer battery policies.
P1 | Calls | Voice is turn-based and depends on Whisper/Piper; it is not a real-time duplex
stream. | Pauses and latency; interruption and natural response may vary. | Design a persistent
audio channel with cancellation, VAD/streaming, and buffers; measure end-to-end latency, loss,
and echo before replacing the current flow.
P1 | Bluetooth audio | Headset input may trigger HFP/HSP and reduce music quality while calls
use the Bluetooth microphone. | Music loses stereo/quality during a call. | Detect
route/profile, offer a separate microphone input, and keep A2DP output when supported; test
PipeWire, headset, and Discord/voice simultaneously.
P1 | Devices | Configuration checks do not physically validate a camera, headset, USB device, or
watch. | UI may report a configured source without useful image/audio. | Show separate states
for configured, enumerated, stream opened, and valid sample; provide diagnostics with timestamp
and selected device.
P1 | Mobile API | Sessions depend on phone, API, TLS/address, and pairing state. Android may
suspend Companion. | Intermittent offline state or calls that do not start in the background. |
Use a single state machine, inexpensive heartbeat, controlled reconnection, layer-specific
errors, and long screen-off tests.
P2 | Chat attachments | 5 files, 6 MiB each, 8 MiB total, and 400,000 characters. | Large
datasets and media do not fit; 1 GB files are unsupported. | Add streaming upload, quota-limited
temporary storage, cancellation, and progress; analyze in chunks and clean temporary files.
Raise limits only after measuring RAM, disk, and attack surface.
P2 | Library | Extraction varies and is limited; binary/scanned formats may

become metadata,
static text, or partial output. | Search may miss content visible to a person. | Show
method/percentage/pages extracted, separate OCR, and reprocessing queue; retain source
references and verify excerpts before responding.
P2 | Mobile AR | ARCore is optional; the current map exists only while the screen is open. |
Rotation/exit may lose mapping; unsupported devices cannot use it. | Offer optional encrypted
local persistence, incremental mapping with budget/expiry, and clear ARCore support status;
measure drift and resource use before enabling by default.
P2 | Camera and vision | The model sees sampled frames; cadence/quality vary, without persistent
recording/tracking. | Fast details may be missed; low light/focus affects responses. | Show
frame age/resolution, adjust frequency/focus, and compare real samples using latency and
accuracy metrics.
P2 | Professor/Engineer | Specialization uses configured models; memory/context remain limited.
| Older details may not be recovered. | Add structured per-conversation summaries, retrievable
memory selection, and context controls; evaluate with long conversations and explicit
references.
P2 | Self-evolution | It creates isolated experiments but does not promote code automatically.
Cycles depend on opt-in systemd setup. | "Start" may not work before services are prepared; a
candidate may be mistaken for an applied change. | Show separate "not installed / stopped /
running / waiting / candidate for review" states; keep diff, rollback, and metrics visible.
P2 | Training | QLoRA requires a contract, approved dataset, holdout, GPU, and measurements. It
does not alter the active model. | It may not start without prerequisites; results do not
guarantee general intelligence. | Add preflight checks and storage/VRAM/energy report,
pause/checkpoint, and baseline evaluation; prohibit promotion without review.
P2 | Email | The feature builds a mailto: draft; it does not send directly through SMTP/API. |
The message may not open if no default mail app exists; sending is manual. | Label it "Prepare
draft"; any future send integration should use official OAuth/API, a secret store, confirmation,
and minimal local records.
P2 | Agenda | Local SQLite with no external sync. | Reminders do not automatically follow
external calendars/devices. | Add ICS import/export and optional OAuth integration, with
conflicts and time zones visible.
P2 | Home Assistant | No connector exists; conversation context does not query home states. |
Sending a full inventory would waste context with many sensors and could still use stale
readings. | Plan an opt-in, local, read-only integration queried on demand by room/domain;
summarize relevant states and bounded history with timestamps. Store secrets locally, allowlist
endpoints, and keep device control out of the first release. See the pending proposal
(HOME_ASSISTANT_INTEGRATION_PENDING.md).

P2 | Plugins | Integrations are native; there is no generic third-party plugin marketplace. | Users cannot install plugins as in ChatGPT's store. | Define versioned API, capability permissions, sandbox, signing, and uninstall before building a marketplace.
P2 | JetBrains MCP | Depends on local server and IDE compatibility. Tools may read/modify a project. | Connection/tool listing may fail; tools may affect the workspace. | Show schemas and plan before invocation, limit timeout/output, restrict workspace, and log consent.
P2 | GitHub | Actions depend on gh, network, permissions, and selected branch; editing has per-file limits. | A repository outside account scope cannot be opened; remote changes need care. | Preview diff, confirm repository/branch/path/SHA, protect against conflicts, and offer backup/rollback.
P2 | Sentinel | Uses ClamAV, selected path, and user permissions. It is not EDR and does not scan every disk/process automatically. | "No detections" does not prove the whole system is clean. | Show scope, exclusions, signature/date, and access errors; schedule only with authorization and test false positives.
P3 | State and observability | Subsystems may fail through external dependencies and return generic messages. | It is hard to tell model, API, permission, codec, network, or device errors apart. | Correlate request ID, stage, and sanitized cause; do not log prompts, files, tokens, or audio by default.
P3 | Package installation | The operation backup is a package inventory, not a full snapshot; package managers have side effects. | "Backup" does not guarantee full system rollback. | Call it an inventory, record command/version, provide manager-specific reversal instructions, and never promise full restoration.

Risks to validate before daily use

1. Local-state integrity: test backup/restore of config, agenda, memory, and library in another user directory.
2. Upgrade: check schema changes and preserve old data before replacing an installation.
3. Continuous audio: verify microphone capture stops when a call ends and output routing does not change unexpectedly.
4. Remote API: verify TLS, authentication, device revocation, request limits, and no accidental public exposure.
5. Android devices: test screen-off, restart, permission revocation, reconnection, rotation, and low battery.
6. Model output: test empty response, timeout, untrusted content, tool calls, and context limit.
7. Installation: verify approval dialog, inventory copy, and actual action before declaring success.

Suggested roadmap

Phase A -- basic reliability

- Add automated smoke tests for startup, API, chat, and microphone shutdown.
- Publish sanitized diagnostics that classify missing dependencies, timeouts, and permission errors.
- Document backup and restoration of local directories.
- Keep the API unreachable outside loopback until TLS/authentication is explicitly configured.

Phase B -- phone and watch

- Establish one connection state machine, heartbeat, and backoff-based reconnection.
- Validate foreground service and battery restrictions across multiple Android manufacturers.
- Test notifications with sensitive content hidden by default, WhatsApp, and permission revocation.
- Separate allowed telemetry from conversation and notification content.

Phase C -- voice, vision, and AR

- Measure latency per stage: capture, transcription, generation, synthesis, playback.
- Add Bluetooth-route tests and ensure music profile is restored after calls.
- Define frame, resolution, VRAM, and temperature budgets for vision.
- Persist an AR map only with opt-in, encryption, storage limit, and clear button.

Phase D -- library and knowledge retrieval

- Report detected format, extraction strategy, skipped pages/parts, and confidence.
- Add small safe fixtures per format; never execute imported content.
- Implement an index with verifiable internal source references and consistent removal of derived text.
- Distinguish metadata, OCR, transcription, and full extraction in the UI.

Phase E -- responsible evolution

- Show the difference between a proposed improvement, experiment, candidate, and installed release.
- Require baseline/holdout and quality, latency, VRAM, and energy metrics.
- Stop if safety, quality, or resource limits regress.
- Keep explicit approval for runtime changes, timer installation, training, and publication.

Phase F -- home context

- First implement an opt-in, local, read-only Home Assistant connector.
- Query by room/domain as needed instead of adding every state to the prompt.
- Summarize only relevant entities, with timestamps, unavailable/stale status, and token limits.
- Treat entity names/attributes as untrusted data and keep tokens out of prompts/logs.
- Cover failures, oversized payloads, and disallowed destinations with synthetic tests.
- Consider control actions only in a later phase, with per-action scopes and explicit confirmation.

Reporting a defect

Include commit/version, OS and app version, feature, prerequisites, minimal steps, expected and observed result, sanitized log, and reproduction frequency. Remove tokens, personal emails, user paths, camera URLs, IP addresses, private images, and conversation content. Report vulnerabilities through the private channel in SECURITY.en.md.

Repository facts that should not be mistaken for gaps

- The project has organized src/, tests/, scripts/, and Android/Wear modules; claims that it has no code/test structure are incorrect.
- There are 37 tests/test_.py files using unittest, plus unit and integration scripts. This does not provide a coverage percentage.
- The actual gap is the lack of GitHub Actions/CI and published coverage measurement. Without a run, do not claim the suite passes or coverage is zero.
- The root README has a quick install and requirements; this documentation adds detailed usage and examples. Screenshots and videos of real flows could still improve it.
- The title, package, and URL consistently use "Vladmir." The received text does not prove it is a typo; only the maintainer can say whether the name was intentional or "Vladimir" was desired.
- The standard MIT license is consistent with publishing Vladmir as open source; "generic" is not a technical defect. It contains full legal terms and the README explains what it permits.
  Third-party components have their own notices.
- Missing governance files were a real gap and have been addressed with contribution guidance, code of conduct, and issue/PR templates.
- The Reddit marketing claim cannot be confirmed from repository files and is not evidence of technical state.

Recommended completion criteria

Mark a fix complete only with evidence at the same level as the issue: static inspection proves a code path; a unit test validates an isolated function; integration validates real services; a physical test validates camera, audio, Bluetooth, or sensors. State test limitations and do not present a mock/fixture as proof of real-device operation.